Sorting Algorithms

Lecture 31

CS 1001

Kirk Duran

Today

- Connect binary search from last lecture to the need for ordered data.
- Understand sorting as the growth of a sorted region.
- Perform selection sort by hand, implement it, and trace the Python code.
- Perform insertion sort by hand, implement it, and trace the Python code.
- Use Python's `.sort()` and `sorted()` for ordinary programs.
- Explain `lambda` and use `key=` to sort lists of objects.

Key idea

First learn how sorting works; then use Python's built-in tools for ordinary programs.

From Last Time: Binary Search Needs Sorted Data

- Of the list-search algorithms we studied, binary search uses fewer checks on large lists when the list is already ordered.
- Binary search can decide which half to discard only because values are in order.
- What happens when the collection we receive is not sorted?



Key idea

Before we can use binary search on an unordered list, we need to put the list in order. Sorting itself also takes work.

Part 1: The Idea of Sorting

What Is Sorting?

Definition

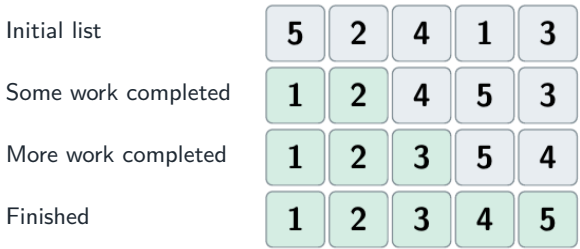
Sorting arranges elements according to an ordering rule, such as increasing numeric value or alphabetical name.

- Numbers can be arranged from smallest to largest.
- Names can be arranged in alphabetical order.
- Objects can be arranged by a selected attribute, such as `student.gpa`.

Our convention today

Both algorithms arrange a list of integers in ascending order and modify that list directly.

Visualizing a Growing Sorted Region



- Green marks the sorted prefix; gray marks the unprocessed region.
- Gold will mark the active value or gap in a trace.

Key idea

Selection sort and insertion sort both grow a sorted prefix, but grow it differently.

Two Ways to Grow the Sorted Region

Selection sort

Find the smallest remaining value and move it to the next position in the sorted region.



Insertion sort

Take the next unsorted value and insert it into the correct position within the sorted region.



Key idea

Selection sort's prefix contains the smallest finished values. Insertion sort's prefix is ordered, but later values may be smaller.

Part 2: Selection Sort

Selection Sort: Example Before Code

Start with `[5, 2, 4, 1, 3]`. Find the smallest value in the whole unsorted region.



- The smallest value is `1` at index `3`.
- Swap that value with the first unsorted position (index `0`).



Key idea

One complete pass puts one more value in its final sorted position.

Selection Sort: Continue the Example



Observation

We never need to examine the finished green positions when searching for the next minimum.

Selection Sort: Describe the Procedure

1. Mark the first unsorted position as `start`.
2. Assume the smallest remaining value is at `start`.
3. Scan the remaining unsorted positions to find the actual minimum.
4. Swap that minimum into position `start`.
5. Move `start` one position to the right and repeat.

Key idea

Selection: repeatedly select the minimum from the remaining values.

Selection Sort: Python Implementation

Sort the existing list in ascending order

```
def selection_sort(values: list[int]) -> None:
    for start in range(len(values) - 1):
        min_index = start
        for scan in range(start + 1, len(values)):
            if values[scan] < values[min_index]:
                min_index = scan
        values[start], values[min_index] = values[min_index], values[start]
```

- `start`: the position we are about to finish.
- `min_index`: the smallest value's index seen during this pass.
- `scan`: checks the remaining candidates.
- The assignment exchanges the two values after the scan ends.

Trace the Code: Selection Sort, First Pass

Relevant code

```
start = 0
min_index = start
for scan in range(1, 5):
    if values[scan] < values[min_index]:
        min_index = scan
```

Variable trace

scan	min_index	Why?
1	1	$2 < 5$
2	1	$4 < 2$ false
3	3	$1 < 2$
4	3	$3 < 1$ false



Key idea

The scan remembers the index of the minimum; it does not swap on every comparison.

Trace the Code: Selection Sort, The Swap

The line that runs after the inner loop

```
values[start], values[min_index] = values[min_index], values[start]
```



- At this moment, `start = 0` and `min_index = 3`.
- The values at positions 0 and 3 exchange places.
- Python evaluates both right-hand values before assigning the left-hand positions.
- The next iteration begins with `start = 1`.

Trace the Code: Selection Sort, Outer Loop

`start`	`min_index`	Action	List after pass
0	3	swap 5,1	`[1,2,4,5,3]`
1	1	same position	`[1,2,4,5,3]`
2	4	swap 4,3	`[1,2,3,5,4]`
3	4	swap 5,4	`[1,2,3,4,5]`

Key idea

`start` also counts the positions already finished. When only one value remains, it must already be in place.

Selection Sort: Check Understanding



1. Which positions are already finished?
2. What value should be selected next from the remaining region?
3. Which two indices should be swapped?
4. What will the list look like after the next pass?

Remember

Choose the minimum from the unsorted suffix, not from the whole list again.

Part 3: Insertion Sort

Insertion Sort: The Playing-Card Idea

- Imagine holding cards in a hand that is already sorted.
- Pick up one more card and slide larger cards to the right to make room.
- Place the new card into the open spot.

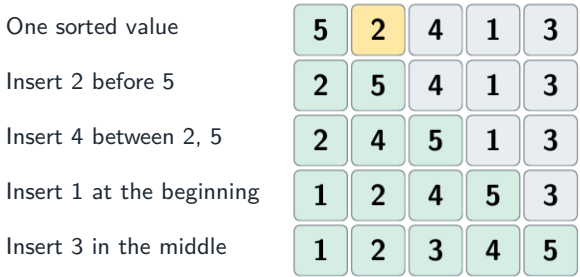


Key idea

Insertion: take the next value and insert it into the correct position within the sorted prefix.

Insertion Sort: Example Before Code

Start with `[5, 2, 4, 1, 3]`. A one-element prefix is already sorted.

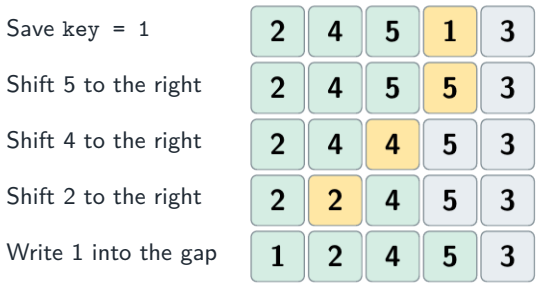


Notice

The prefix is ordered after every insertion, but need not contain the smallest values in the entire list yet.

Insertion Sort: What Does “Insert” Mean?

Suppose the list is `[2, 4, 5, 1, 3]` and the saved key is `1`.



Repeated numbers during shifting are temporary: the key is safely stored in a variable. Gold tracks the active slot or gap.

Insertion Sort: Describe the Procedure

1. Start with the first element as a one-value sorted region.
2. Save the next value in a variable called `key`.
3. Walk left through the prefix while values are larger than `key`.
4. Shift each larger value one position to the right.
5. Write `key` into the gap and grow the sorted region.

Key idea

Insertion sort makes room by shifting, rather than finding the overall minimum.

Insertion Sort: Python Implementation

Sort the existing list in ascending order

```
def insertion_sort(values: list[int]) -> None:
    for position in range(1, len(values)):
        key = values[position]
        scan = position - 1
        while scan >= 0 and values[scan] > key:
            values[scan + 1] = values[scan]
            scan -= 1
        values[scan + 1] = key
```

- `position` identifies the next item to insert; `key` holds it safely.
- `scan` moves left through the sorted prefix.
- and tests `scan >= 0` first; when false, it skips `values[scan]`.

Trace the Code: Insertion Sort, First Key

Relevant code

```
position = 1
key = values[position]
scan = position - 1
while scan >= 0 and values[scan] > key:
    values[scan + 1] = values[scan]
    scan -= 1
values[scan + 1] = key
```

Values and assignments

Initial	`[5,2,4,1,3]`
key	2
scan	0
Shift 5	`[5,5,4,1,3]`
`scan -= 1`	-1
Insert 2	`[2,5,4,1,3]`

Key idea

The extra copy of 5 is temporary. When the loop ends, the saved key fills the gap.

Trace the Code: Insertion Sort, A Larger Shift

Take `key = 1` at `position = 3` in `[2, 4, 5, 1, 3]`.

scan	Compare	Assignment	List afterward
2	$5 > 1$	`values[3] = 5`	`[2,4,5,5,3]`
1	$4 > 1$	`values[2] = 4`	`[2,4,4,5,3]`
0	$2 > 1$	`values[1] = 2`	`[2,2,4,5,3]`
-1	stop	`values[0] = 1`	`[1,2,4,5,3]`

Key idea

When `scan` reaches -1, the saved key belongs at index `0`.

Trace the Code: Insertion Sort, Outer Loop

position	key	Shifts	List after insertion
1	2	5	[2,5,4,1,3]
2	4	5	[2,4,5,1,3]
3	1	5,4,2	[1,2,4,5,3]
4	3	5,4	[1,2,3,4,5]

Key idea

Each iteration inserts one new key into a prefix that was already ordered.

Insertion Sort: Check Understanding



1. Which three values are in the sorted prefix?
2. What value is saved in `key`?
3. Which existing values must shift right?
4. At what index is `key` finally inserted?

Remember

`key` is saved before any overwriting begins.

Selection Sort vs. Insertion Sort

	Selection sort	**Insertion sort**
Next step	Find smallest value in unsorted suffix	Take next value and move it left into order
Movement	One exchange after scanning	Shift larger values, then place key
Sorted region	Contains smallest finished values	Contains values seen so far, in order
Changes input?	Yes	Yes

Key idea

Both methods help us understand sorting. In everyday Python code, use built-in sorting.

Part 4: Python Sorts for You

The List Method: `.sort()`

Modify the original list

```
values = [5, 2, 4, 1, 3]
values.sort()
print(values)  # [1, 2, 3, 4, 5]
```

- `.sort()` belongs to a list object.
- It rearranges the existing list in place.
- It returns `None`, not the sorted list.

Common mistake

`result = values.sort()` makes `result` equal to `None`.

The Function: `sorted()`

Create a new sorted list

```
values = [5, 2, 4, 1, 3]
ordered = sorted(values)
print(ordered)  # [1, 2, 3, 4, 5]
print(values)   # [5, 2, 4, 1, 3]
```

- `sorted(values)` returns a new list; the input list remains unchanged.
- `sorted()` also accepts other iterables, such as a tuple or string.
- With objects, the new list still holds references to the original objects.

Key idea

Use `.sort()` to change the original; use `sorted()` for a separate list.

Descending Order: `reverse=True`

Both sorting interfaces accept the option

```
values = [5, 2, 4, 1, 3]
values.sort(reverse=True)
print(values)  # [5, 4, 3, 2, 1]

ordered = sorted([5, 2, 4, 1, 3], reverse=True)
print(ordered)  # [5, 4, 3, 2, 1]
```

- `reverse=True` requests descending order.
- `values.reverse()` just flips the existing sequence; it does not sort it.

Which Should I Use?

Goal	Python expression
Reorder the existing list	`values.sort()`
Make a new ordered list	`sorted(values)`
Reorder existing list, descending	`values.sort(reverse=True)`
Make a descending copy	`sorted(values, reverse=True)`

Key idea

You usually do not need to implement selection or insertion sort to sort real Python data.

Part 5: Sorting Objects with `lambda`

A List of Objects Needs an Ordering Rule

The Student class from last lecture

```
class Student:
    def __init__(self, student_id, name, gpa):
        self.student_id = student_id
        self.name = name
        self.gpa = gpa
```

- A student has several attributes: ID, name, and GPA.
- Should students be ordered by ID? Alphabetically? By GPA?
- Python cannot infer which attribute we mean from this simple class.

Key idea

For object sorting, the programmer supplies the key used for comparisons.

The `key=` Argument

Sort names by their length

```
names = ["Ada", "Christopher", "Bo"]
ordered = sorted(names, key=len)
print(ordered)  # ['Bo', 'Ada', 'Christopher']
```

- `key=` receives a function, not a precomputed result.
- Python calls that function once per item to obtain its sorting key.
- The return value determines the ordering: `len("Bo")` is 2.

Key idea

`key=` specifies what property to compare, not how to write a sorting loop.

What Is a `lambda`?

Definition

A `lambda` expression creates a small function without giving it a name using `def`.

Named function

```
def get_gpa(student):
    return student.gpa
```

Equivalent lambda

```
lambda student: student.gpa
```

Key idea

A lambda receives input and evaluates one expression, whose result is returned automatically.

Read `lambda student: student.gpa` Piece by Piece

```
lambda student: student.gpa
```

Piece	Meaning
`lambda`	create a function value
`student`	parameter: one input object
`:`	separates input from the expression
`student.gpa`	value returned for that object

Example evaluation

Called with `Student(1042, "Luis", 3.8)`, this function returns `3.8`.

Sort Student Objects by GPA

The input list

```
students = [
    Student(1008, "Maya", 3.4),
    Student(1042, "Luis", 3.8),
    Student(1091, "Nora", 3.6),
]
```

Sort by the GPA attribute

```
by_gpa = sorted(students, key=lambda s: s.gpa)
```

Resulting order: Maya (3.4) $\rightarrow$ Nora (3.6) $\rightarrow$ Luis (3.8)

How Does `key=lambda s: s.gpa` Work?

Object	Key returned	Sorted position
`Student(1008, "Maya", 3.4)`	3.4	first
`Student(1042, "Luis", 3.8)`	3.8	third
`Student(1091, "Nora", 3.6)`	3.6	second

- The function receives each `Student` object as `s`.
- It returns that student's GPA as the comparison key.
- The result contains the original objects, not just GPAs.

Key idea

Changing the key changes the ordering rule without changing the objects.

Different Keys, Different Orders

Same list of Student objects

```
# Alphabetical by name, new list
by_name = sorted(students, key=lambda s: s.name)

# Highest GPA first, original list changes
students.sort(key=lambda s: s.gpa, reverse=True)

# Increasing student ID, new list
by_id = sorted(students, key=lambda s: s.student_id)
```

- `key=` chooses the attribute; `reverse=True` reverses the order.
- `.sort()` versus `sorted()` determines whether the input changes.
- Equal keys keep their original relative order: Python sorting is **stable**.

Connect `lambda` to Last Lecture

Last lecture: search predicate

```
lambda s:  s.gpa >= 3.8
```

Returns `True` or `False`: does this object match?

Today: sorting key

```
lambda s:  s.gpa
```

Returns a GPA number: what value determines this object's order?

Key idea

The lambda syntax is identical; the function's purpose and returned value differ.

Common Mistakes with Object Sorting

1. `key=lambda s: s.gpa()` is wrong if gpa is an attribute, not a method.
2. `key=lambda s: s.gpa > 3.5` groups by a Boolean key; it does not order by GPA within those groups.
3. `key=lambda s: s` asks Python to compare whole objects, which our basic `Student` class does not support.
4. `students = students.sort(...)` replaces the variable with `None`.

Key idea

A good key returns exactly the property you want the elements sorted by.

Integrated Practice: Predict the Order

Suppose `students` contains Maya (3.4), Luis (3.8), and Nora (3.6).

1. Who comes first after `sorted(students, key=lambda s: s.name)`?
2. Who comes first after `sorted(students, key=lambda s: s.gpa)`?
3. Who comes first with `key=lambda s: s.gpa, reverse=True`?
4. Which changes the original list: `.sort()` or `sorted()`?
5. Does the result contain `Student` objects or just numbers and names?

Takeaways

1. Binary search relies on ordered data; sorting can provide that order.
2. Both algorithms grow a sorted region from left to right.
3. Selection sort finds the minimum remaining value and swaps it into place.
4. Insertion sort saves the next key, shifts larger values, and inserts the key.
5. `.sort()` changes the original list; `sorted()` returns a new list.
6. `reverse=True` requests descending order.
7. `lambda` creates a short function; `key=lambda s: s.gpa` orders by GPA.

Key idea

Understand the algorithms; use built-in sorting in practice.